MindWeave-R: Contract-Aware Detection and Budgeted Repair of Artifacts in Multi-Agent Workflows

Minal Barhate, [Author 2], [Author 3]
Vishwakarma Institute of Technology, Pune, Maharashtra, India
[author emails to be added]

***Abstract*—**Multi-agent workflows chain language-model agents through a directed graph, and every connection silently assumes something about the data crossing it: that fields exist, that confidence is high enough, that provenance is intact, that the payload is fresh and within budget. When an assumption breaks, most orchestrators either continue with degraded data or abort the run. We present MindWeave-R, a system in which each edge carries an executable contract and a failed check triggers a budgeted repair decision instead of termination. On failure the engine classifies the violation into one of six codes, generates the repair operators that cover every violated code, selects among budget-feasible candidates by a weighted cost, latency, privacy and residual-risk score, applies the repair to the artifact, and commits a Merkle-rooted certificate over the trace. We evaluate a seven-node research workflow under 180 synthetic fault injections across six classes. All 180 were detected, and every repaired artifact satisfied its contract when re-checked, at 4.5 to 18.2 percent credit overhead, with no repair on 30 healthy control runs. Disabling the gate exposed the violation in all six classes and every violating artifact was consumed downstream while the run still reported completion. Three negative findings matter more than the positive one: single-code faults always admit exactly one candidate, so the ranking rule is never exercised by the deployed catalogue; compound violations admit no candidate at all; and 'repaired' does not mean 'valid', because five of our six operators leave the payload untouched and the sixth inserts a placeholder. We argue that contract satisfaction must be re-checked after repair, and we report the places where our own implementation does not.

***Index Terms*—**Multi-agent systems, workflow orchestration, data contracts, automated repair, provenance, auditability.

I. INTRODUCTION

Multi-agent workflows have become a standard way to build applications on top of large language models. A reasoning agent gathers evidence, a second validates it, a third drafts, a fourth checks the draft, and a human approves the result before release [1], [2], [3]. Canvas-based orchestrators have followed, letting a user wire such agents into a directed acyclic graph and run it, which makes the composition itself the object of design rather than the prompt.

Composition introduces a failure mode that single-agent work does not have. Each connection carries assumptions about the data crossing it: that a field exists, that confidence exceeds some threshold, that the payload is recent, that provenance is intact, that it is not too expensive, and that it is not classified more sensitively than the receiving side may handle. In most systems none of these assumptions is expressed where the runtime can check it. When one breaks, the artifact moves on. A recent empirical taxonomy of multi-agent failures places verification and inter-agent misalignment among the dominant causes [4], and role-based agent pipelines report hallucination cascading through unverified intermediate outputs [3].

Systems research offers two adjacent answers, and neither fits. Orchestrators handle infrastructure faults such as lost workers, timeouts and host failure with retries, replication and rescheduling [5], [6]; but a task that completes successfully and emits unusable data is not a fault in that sense, and retrying it reproduces the same artifact. Data-validation systems evaluate expectations at pipeline boundaries [7], [8] and can learn constraints from execution history [9]; but they report and halt, leaving the next decision to a person.

The repair literature is mature in three neighbouring domains. Automated program repair localises a fault, generates candidate patches and validates against an oracle [10], [11]. Plan repair restores a plan that has become infeasible, including under numeric resource constraints [12]. Process-model repair aligns a model with observed behaviour [13], [14]. All three share the pipeline shape we adopt; none operates on a runtime data artifact at a graph edge, and none selects a remedy under a hard spend budget with privacy exposure and residual failure risk as competing costs.

This paper presents MindWeave-R, a workflow system in which connections are executable contracts and a failed check produces a budgeted repair decision rather than a termination. Our contributions are:

(C1) a formulation of edge contracts as first-class runtime predicates with a six-code failure taxonomy, evaluated against the artifact rather than against the agent;

(C2) a repair pipeline, comprising code classification, coverage-filtered candidate generation, budget-feasible weighted selection and artifact-level application that preserves acyclicity by construction, in which the selection objective trades spend against latency, privacy exposure and residual failure risk;

(C3) an audit certificate that commits, under a single Merkle root, to the execution trace, the repair decisions and the final artifact, making the decision replayable rather than only the outcome inspectable;

(C4) an empirical characterisation over 180 injected faults in six classes, including a gate ablation, a healthy control, and two negative results that bound what the system actually establishes.

We are explicit about scope. We do not propose a new repair algorithm, and we claim no novelty in ranking, in hash trees or in approval gates. Our claim is about the combination and about where the decision boundary is drawn. Sections VII and VIII report where our implementation falls short of its own design.

II. RELATED WORK

A. Workflow Orchestration and Fault Tolerance

Workflow management systems separate the description of a computation from its execution, which is what allows an abstract

graph to be mapped onto available resources; Pegasus has done so in production for two decades [5]. Production machine-learning platforms extend the model to analysis, validation and serving stages [15]. Fault tolerance in this tradition is about availability: replication, checkpointing and dynamic rescheduling keep a workflow alive when infrastructure fails [6].

Our faults differ in kind. The node succeeds, the edge is traversed, and what crosses it is wrong. No amount of retrying discovers this, because the retry is deterministic in the same way. This is why we locate the check at the edge rather than at the scheduler, and why uptime and completion rate are the wrong instruments for measuring it.

B. Data Validation and Contracts

Declarative, expectation-based validation is well established. Deequ computes data-quality constraints at scale and frames them as unit tests for data [7]; TensorFlow Data Validation infers and enforces schemas at pipeline boundaries [8]; Auto-Validate learns validation programs from the history of recurring pipelines [9]. Schema evolution has its own body of work, of which PRISM is a canonical system [16], and provenance capture for reproducibility is addressed by systems such as Ursprung [17].

What this literature does not provide is a decision procedure after detection, and constraints are generally attached to a pipeline rather than to a connection. The distinction matters for agent graphs: two edges leaving the same node may legitimately require different confidence thresholds, and it is the edge, not the pipeline, that carries the assumption.

C. Reliability of LLM Agents

Agent design has been improved from several directions: the reason-act-observe loop [1], conversational multi-agent frameworks [2], role-specialised pipelines with standardised operating procedures [3], self-critique [18], [19], and critique grounded in external tools [20]. Risk evaluation for tool-using agents has been studied in emulated environments [21]. The MAST taxonomy [4] is the most direct evidence that verification is where these systems break, grouping fourteen failure modes into system design, inter-agent misalignment and task verification.

Almost all of this work improves the agent. We take agents as given and constrain their handoffs, which makes the approach composable with any of it. It is also why our checker does not ask the agent to assess itself: self-assessment has repeatedly been shown weaker than external verification [20].

D. Automated Repair

The structure of program repair, localise then generate then validate, is the template we follow [10], and its literature is a warning as much as a guide: a patch that satisfies the oracle may still be wrong, which is the central open problem of the field [11]. Plan repair under numeric resource constraints [12] is the closest formal precedent for our budget constraint. Process-model repair operates offline against event logs [13], [14] and addresses structure rather than data.

We borrow the pipeline and the caution. Our repair acts on a single artifact, the candidate set is derived from the violated codes rather than from a search over patches, and the objective combines four costs under one hard constraint.

E. Provenance, Audit and Oversight

PROV supplies the vocabulary of entities, activities and agents that we deliberately simplify [22]. Tamper-evident logging by hash chaining is old and well understood, including its limits: without a trusted time source or a key, a hash chain establishes integrity and nothing more [23]. On oversight, Green shows that mandating a human in the loop does not by itself produce meaningful review [24], and Cobbe *et al.* frame reviewability as the property that makes a decision interrogable after the fact [25].

We take the last point as a design requirement: what should be committed is not only the output but the reason a particular remedy was chosen.

F. Positioning

Three gaps follow. **G1:** validators detect but do not decide, and enforcement is a deployment policy rather than a specified procedure at runtime and per connection. **G2:** repair exists for code, plans and process models, but no prior work selects among artifact-level repair operators under a hard spend budget together with privacy and residual-failure risk. **G3:** provenance and tamper-evident logging exist separately from repair, so the *choice* of remedy is not typically auditable. MindWeave-R addresses the three as one mechanism; nothing below should be read as a claim that any single component is new.

III. PROBLEM FORMULATION

Definition 1 (workflow). A workflow is a pair $W = (V, E)$ where E is a subset of $V \times V$ and (V, E) is a directed acyclic graph. Each node v has a kind $k(v)$ drawn from a fixed catalogue, and each kind declares an output field set $O(k)$.

Definition 2 (artifact). An artifact is a tuple $a = (\text{kind}, \text{payload}, \text{prov}, \text{conf}, \text{priv}, t, \text{cost}, h)$, where payload is a mapping from field names to values, prov is a list of lineage tokens, conf is in $[0, 1]$, priv is one of public, internal or confidential with the order $0 < 1 < 2$, t is a creation timestamp, cost is expressed in credits, and h is a SHA-256 digest over the canonical JSON encoding of the other seven members.

Definition 3 (contract). Each edge $e = (u, v)$ carries a contract $c = (F, \tau, \pi, \Delta, B, \rho)$ where F is a set of required field names, τ a minimum confidence, π a maximum privacy classification, Δ a freshness window in seconds, B a cost ceiling in credits, and ρ a Boolean requirement that provenance be non-empty.

Definition 4 (violation). An artifact a violates c if its code set is non-empty, where

$$\text{codes}(a, c) = \{\ \text{schema if } F \not\subseteq \text{fields(payload)};\ \text{confidence if conf} < \tau;\ \text{privacy if priv}(a) > \pi;\ \text{freshness if now} - t > \Delta;\ \text{budget if cost}(a) > B;\ \text{provenance if } \rho \text{ and prov is empty}\ \}.$$

Definition 5 (repair operator). An operator is a tuple $o = (\text{id}, \kappa, \lambda, p, r, X)$, where κ is its cost in credits, λ a latency class, p a privacy-exposure risk, r a residual failure risk, and X a non-empty subset of codes it is declared to fix. Operator o covers a violation with code set K if $K \subseteq X$, and is feasible if $\kappa(o)$ does not exceed the remaining budget.

Problem (repair selection). Given a violating artifact a, a contract c and a remaining budget, choose

$$o^* = \arg\min_{\text{feasible covering } o} \alpha\cdot\kappa(o) + \beta\cdot\lambda(o) + \gamma\cdot p(o) + \delta\cdot r(o)$$

and if no feasible covering operator exists, block the edge. The four weights encode a deliberately asymmetric loss: blocking is safe but consumes human attention, whereas repairing is cheap but injects risk, so r is weighted above κ by a factor of six. Our implementation uses (α, β, γ, δ) = (1, 0.2, 4, 6). These are design weights, not a calibrated utility model, and we do not claim they are optimal.

Definition 6 (certificate). A run produces leaves L = {H(x) for x in trace} plus {H(y) for y in repairs} plus {h_final}. The certificate root is the Merkle reduction of L under pairwise SHA-256, duplicating the final leaf when the layer length is odd.

IV. METHOD

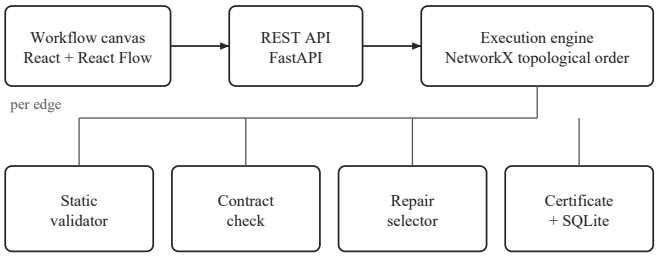


Fig. 1. System structure. The canvas issues REST calls; the engine orders nodes topologically and, per edge, invokes the static validator once per graph and the contract check, repair selector and certificate builder per completed artifact.

A. Contracts on Edges

A contract belongs to the connection, not to the node, because the handoff is where the assumption lives. The same artifact may be checked against several outgoing contracts with different thresholds, and may satisfy one while violating another; that case is handled by repairing per edge, so the downstream node receives an artifact conforming to the contract it actually traversed.

Before execution, a static pass rejects graphs that are cyclic, that reference a missing endpoint, that carry a malformed contract, that require a field the source cannot produce, whose edge privacy exceeds the receiving node's declared maximum, or in which some output node is not reachable from an approval node. Confidence, freshness and budget are properties of a concrete artifact and are therefore not statically decidable; they are enforced only at runtime.

B. Detection

After a node completes, its artifact is tested against each outgoing contract. The six predicates are evaluated independently and reported together, so one artifact can carry several codes at once. This matters for candidate generation below, and it is where our design turns out to be weakest.

C. Candidate Generation

The pool is every operator whose fix set is a superset of the violated codes. The coverage rule is intentional: it prevents a partial remedy from being applied to a compound violation and then reported as a repair. Its consequence is that a violation whose codes no single operator covers yields an empty pool and blocks the run. Section VII-D measures exactly this case.

D. Selection

Feasible candidates are ordered by the weighted score of Section III. Ties break on catalogue order rather than at random, because a deterministic choice is what makes a certificate replayable: the same artifact, contract and budget must always select the same operator.

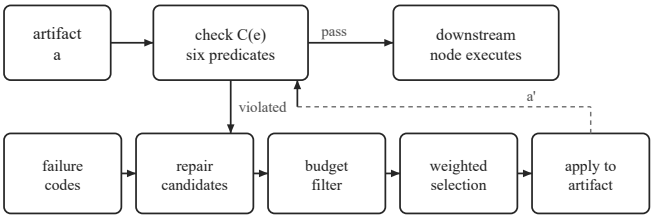


Fig. 2. Per-edge repair loop. A completed artifact is checked against the outgoing contract; on failure the codes select operators from the catalogue, the budget filter removes unaffordable ones, the weighted rule picks one, and the repaired artifact a' is what the downstream node consumes (dashed path). The certificate commits to the whole sequence.

E. Application

A repair rewrites the artifact, appends a lineage token naming the operator, recomputes the digest and passes the result downstream. Because no node or edge is added, the graph remains acyclic: the DAG invariant is preserved by construction rather than re-established by a reachability check. The alternative, inserting a repair subgraph, would require such a check before the artifact could be routed, and we do not implement it; our operators are artifact-local.

F. Certificate

Each completed run with at least one repair stores a certificate: the ordered trace hashes, the repair records (before hash, after hash, codes, chosen operator, cost) and the final artifact hash, reduced to one root. The encoding is canonical JSON with sorted keys and no insignificant whitespace, so independent reimplementation yields the same root.

What the root establishes is narrow, and we state it in the same terms as the audit-log literature [23]: it proves that the recorded sequence has not been altered since it was hashed. It does not prove authenticity, because nothing is signed; it does not prove completeness, because a run may simply record fewer events; and it says nothing about whether the repair was correct. It is a record, not a warranty.

G. The Approval Gate

Independently of repair, the engine halts before any final output and waits for an explicit approval or rejection, recorded as an execution status change. This is a structural policy, enforced as reachability in the static pass. We note Green's argument [24] that a mandated human step is not by itself meaningful oversight, and we therefore claim only that our gate produces an auditable pause, not that it produces good review.

V. IMPLEMENTATION

The engine and HTTP API are a single Python module of about 230 lines built on FastAPI and Pydantic v2, with NetworkX for topological ordering and SQLite for users, workflows, executions, artifacts and certificates. The client is a React and TypeScript canvas on React Flow with thirteen node kinds, a palette, an inspector for node guardrails and edge contracts, a trace strip and a

wallet panel. Module nodes nest a saved workflow and recurse through the same engine with approvals disabled, which is what allows workflows to be composed hierarchically.

Two implementation choices shape the evaluation. First, a wallet meter: each node kind has a credit price, a run is refused when the balance cannot cover the nominal cost, and a repair is charged on top. Second, a deterministic demo mode in which every agent emits synthetic, stable payloads and no model is called. Demo mode is what makes the fault-injection experiments below exactly reproducible by hand or by script; it is also the largest threat to the validity of those experiments, and we return to this in Section VIII.

All numbers reported here are produced by a harness of about 300 lines that imports the engine, injects faults, and writes the result tables; it runs in seconds with no network access. The seven unit tests shipped with the system cover authentication, workflow ownership, static validation, topological ordering, repair planning, certificate persistence and Merkle sensitivity.

VI. EXPERIMENTAL SETUP

We organise the study around four hypotheses, each of which our design can falsify:

H1. The gate detects and repairs every single-code fault class it defines.

H2. With the gate disabled, a violating artifact reaches downstream agents and the run still completes.

H3. Single-code faults admit exactly one candidate, and compound faults admit none.

H4. The weighted rule differs from cost-only, risk-only and order-first baselines on the same pool.

The subject is the Evidence Research template: seven nodes and six edges, from topic input through researcher, curator, validator, writer and approval to final output, at a nominal cost of 11 credits against a wallet of 100. A fault is injected at the researcher's output by a backdoor field on the node, and the contract on the researcher to curator edge is tightened so that the injected condition is the condition violated. The six classes are: low confidence (0.10 against a threshold of 0.80); a dropped required field; emptied provenance with provenance required; a confidentiality downgrade from the artifact's classification to a public contract; a stale timestamp against a one-second window; and an artifact cost above a contract ceiling.

Each class is run 30 times, for 180 injections in total, and a further 30 healthy runs form a control. Because the agents are deterministic, all repetitions of a class are byte-identical; the harness asserts this. We therefore report per-class outcomes rather than distributional statistics, and 180 injections are six behaviours repeated, not 180 independent samples. No significance test would be meaningful here and none is reported.

VII. RESULTS AND DISCUSSION

Table I reports the per-class outcome. Every injection was detected with exactly the expected code, the corresponding operator was selected, and the artifact satisfied its contract when re-checked immediately after repair. Repair cost ranged from 0.5 to 2 credits against a nominal workflow cost of 11, an overhead of 4.5 to 18.2 percent, and the total charge per repaired run moved from 11 to 12-16 credits.

TABLE I

Per-class outcome over 30 injections per class. All repetitions within a class were identical. “Post” is the contract check applied to the repaired artifact; “Payload” records whether the payload content changed at all.

Fault class	Selected operator	Post	Payload	Credits	Overhead
confidence	consensus-check	yes	no	16	18.2%
schema	schema-enricher	yes	yes	15	9.1%
provenance	source-curator	yes	no	15	9.1%
privacy	privacy-redactor	yes	no	15	9.1%
freshness	freshness-refresh	yes	no	16	18.2%
budget	context-compressor	yes	no	12	4.5%

The aggregate figure, six of six classes repaired, invites a stronger conclusion than the data supports, and three observations argue against it.

Finding 1 (degenerate selection). The candidate pool contained exactly one operator in every single-code case. The catalogue maps each code to one operator, so the coverage rule leaves the ranking rule nothing to rank, and Table I is a table of detections rather than of decisions. Contribution C2's selection stage is implemented and unit-tested on constructed pools, but the deployed catalogue never exercises it. This is a property of our operator set, not of the formulation; a realistic catalogue with several operators per code, or with multi-code operators, would make the score decisive.

Finding 2 (satisfaction is not validity). Re-checking the repaired artifact succeeded in all six classes, yet the payload changed in only one. The schema class is satisfied by inserting the literal string “repaired:evidence” in place of the dropped field. The confidence class is satisfied by assigning $\tau + 0.05$, moving the artifact from 0.10 to 0.85 without any new evidence being gathered. The freshness operator rewrites the timestamp and leaves the content untouched. A presence-based schema predicate cannot distinguish a real field from a placeholder, and a threshold predicate cannot distinguish measured confidence from asserted confidence. This is the program-repair lesson [10], [11] reappearing one level up: passing the oracle is not being correct, and our oracle is weaker than a test suite.

Finding 3 (the gate does not change completion). Table II reports the ablation. With enforcement disabled, the violation was detected by the same checker, the violating artifact was consumed by the next agent, and the run completed. With enforcement enabled, no violating artifact reached an agent, and the run still completed. Completion rate, the metric most commonly reported for orchestration systems, is identical in both conditions and therefore blind to the mechanism: the gate is an integrity barrier, not a liveness barrier, and a study measuring only success rate would conclude, wrongly, that contracts do nothing.

TABLE II

Gate ablation. “Consumed”: the violating artifact is used as input by a later node.

Fault class	Off: consumed	Off: repairs	On: consumed	On: repairs	Run completes
confidence	yes	0	no	1	yes
schema	yes	0	no	1	yes
provenance	yes	0	no	1	yes
privacy	yes	0	no	1	yes
freshness	yes	0	no	1	yes
budget	yes	0	no	1	yes

Finding 4 (compound violations are unreachable). Combining two codes on one artifact produced an empty pool in all three configurations tested, and execution stopped with a contract-repair error rather than proceeding (Table III). Combined with Finding 1, this means the system can repair exactly the violations for which it needs no decision, and cannot repair the ones where a decision would matter. The cause is structural: the coverage rule is a subset test and each operator fixes a single code. The obvious repairs are partial coverage with a residual code set that triggers a second round, or a compositional operator that is itself a small repair graph.

TABLE III

Compound violations. The pool is empty because no operator's fix set covers both codes.

Violated codes	Candidates	Outcome
confidence + schema	0	repair unavailable
confidence + provenance	0	repair unavailable
privacy + schema	0	repair unavailable

Finding 5 (the rule discriminates when it is given a choice). To exercise the score at all we constructed a pool of five multi-code operators with deliberately different cost and risk profiles and compared the proposed rule with three baselines (Table IV). The weighted rule selects an operator at 2.4 times the cost of the cheapest but with 30 percent of its residual risk; the cost-only baseline takes the cheap option at 0.30 residual failure risk; the risk-only and order-first baselines both take the most expensive option. The behaviour matches the intent of the weights, and the pool is constructed, not sampled from a real catalogue, so this is evidence that the rule is implemented as specified, not that it is a good rule.

TABLE IV

Selection rules on a constructed five-operator pool. All rules operate on the same candidates; only the choice differs. Score is the weighted objective at the selected point.

Rule	Selected operator	Cost	Residual	Score
weighted	privacy-scrubbing	1.2	0.09	2.14
least cost	cheap-repair	0.5	0.30	2.50
least residual risk	thorough-recompute	3.0	0.02	3.52
first feasible	thorough-recompute	3.0	0.02	3.52

Finding 6 (cost is not the obstacle). 30 healthy control runs produced 0 repairs and 0 violations at 11 credits, equal to the nominal workflow cost. Repair overhead is between 4.5 and 18.2 percent, and the most expensive repairs are the two that plausibly require work, recomputation and refetching. The mechanism is cheap enough that the interesting question is not its price but its correctness, which is what Findings 2 and 4 are about.

VIII. THREATS TO VALIDITY

Construct validity. The agents are deterministic stand-ins that emit synthetic payloads, so a repair operator that restores information and one that merely rewrites metadata are indistinguishable from the outside. Our operators are almost entirely of the second kind: five of the six leave the payload untouched and the sixth inserts a placeholder. The measured quantity is therefore "contract satisfaction after repair", and we have shown independently in Finding 2 that it over-reports validity. A second, independent validator, ideally one that inspects content rather than declarations, is required before any claim about repair quality.

Circularity. The repaired artifact is re-checked by the same predicate that motivated the repair. A predicate can be satisfied by construction, as the schema case demonstrates. We report this rather than correcting for it, because the correction is exactly the future work.

Internal validity. Faults enter through a backdoor field on the node that is not part of the production path, so the injected conditions are cleaner than real failures, which tend to be partial and correlated across a payload. The single template, seven nodes and one graph topology remove structural variation. Determinism removes variance and therefore also removes the possibility of estimating it; we state per-class behaviour instead of averages over draws.

External validity. One workflow shape, a small node catalogue, no real model calls and no human participants. Nothing here licenses a claim about behaviour under production LLM agents, whose outputs vary between calls, or about the thirteen-node palette beyond the six kinds exercised.

Conclusion validity. We report no comparison against a baseline system, no user study and no effect size, and we make no claim of improvement over the tools in Section II. The contribution demonstrated is that the mechanism operates as specified and that two specific design flaws are real and measurable.

One methodological point deserves emphasis. Our first instrumentation recorded only whether a run succeeded, and by that measure the system repaired 180 of 180 faults with no exceptions. The caveats in Finding 2 exist only because we added a second, post-repair audit. A paper that stopped at the completion counter would have reported a cleaner and false result.

IX. CONCLUSION AND FUTURE WORK

We described a workflow system in which every connection carries an executable contract, a violated contract triggers a repair decision rather than a termination, and the decision is committed to an audit certificate. The mechanism works as specified on the six fault classes it defines, at a credit overhead of under one fifth of the nominal run cost, and is invisible to completion-rate measurement because it does not change whether a run finishes.

The reported negative results define the next steps more sharply than the positive one. First, replace the simulated operators with real ones: re-execution with a different agent, retrieval refresh for staleness, and genuine schema enrichment, so that a repair can restore information rather than assert it. Second, re-validate after repair inside the engine, not only in the harness, and use a validator independent of the contract that triggered the repair. Third, relax coverage to partial coverage with a residual code set, which makes compound violations reachable. Fourth, enlarge the catalogue so the selection rule has something to select. Fifth, sign the certificate, turning an integrity record into an attributable one. Beyond the mechanism, the natural evaluation is the one we have not run: a within-subject comparison against an unconstrained canvas, measuring first-run success, invalid connections caught before execution, time to repair, unnecessary agent calls and user trust, with more than one template and real agents.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[2] Q. Wu, G. Bansal, J. Zhang, *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[3] S. Hong, M. Zhuge, J. Chen, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.

[4] M. Cemri, M. Z. Pan, S. Yang, *et al.*, "Why do multi-agent LLM systems fail?" in *Proc. NeurIPS Datasets and Benchmarks*, 2025.

[5] E. Deelman, K. Vahi, M. Rynge, R. Mayani, R. Ferreira da Silva, G. Papadimitriou, and M. Livny, "The evolution of the Pegasus workflow management software," *Computing in Science & Engineering*, vol. 21, no. 4, pp. 22-36, 2019.

[6] Z. Li *et al.*, "Real-time and dynamic fault-tolerant scheduling for scientific workflows in clouds," *Information Sciences*, vol. 568, pp. 13-39, 2021.

[7] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endow.*, vol. 11, no. 12, pp. 1781-1794, 2018.

[8] E. Caveness, P. Suganthan G. C., Z. Peng, N. Polyzotis, S. Roy, and M. Zinkevich, "TensorFlow Data Validation: Data analysis and validation in continuous ML pipelines," in *Proc. ACM SIGMOD*, 2020, pp. 2793-2796.

[9] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, S. Han, D. Zhang, and S. Chaudhuri, "Auto-Validate by-History: Auto-program data quality constraints to validate recurring data pipelines," in *Proc. 29th ACM SIGKDD*, 2023, pp. 4991-5003.

[10] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56-65, 2019.

[11] Q. Zhang, C. Fang, Y. Ma, *et al.*, "A survey of learning-based automated program repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, art. 55, 2024.

[12] E. Scala, "Plan repair for resource constrained tasks via numeric macro actions," in *Proc. ICAPS*, vol. 24, no. 1, 2014, pp. 280-288.

[13] D. Fahland and W. M. P. van der Aalst, "Model repair - aligning process models to reality," *Information Systems*, vol. 47, pp. 220-243, 2015.

[14] A. Polyvyanyy, W. M. P. van der Aalst, A. H. M. ter Hofstede, and M. T. Wynn, "Impact-driven process model repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 25, no. 4, art. 28, 2016.

[15] D. Baylor *et al.*, "TFX: A TensorFlow-based production-scale machine learning platform," in *Proc. 23rd ACM SIGKDD*, 2017, pp. 1387-1395.

[16] C. A. Curino, H. J. Moon, and C. Zaniolo, "Graceful database schema evolution: the PRISM workbench," *Proc. VLDB Endow.*, vol. 1, no. 1, pp. 761-772, 2008.

[17] L. Rupprecht, J. C. Davis, C. Arnold, Y. Gur, and D. Bhagwat, "Improving reproducibility of data science pipelines through transparent provenance capture," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3354-3368, 2020.

[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.

[19] A. Madaan, N. Tandon, P. Gupta, *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Proc. NeurIPS*, 2023.

[20] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *Proc. ICLR*, 2024.

[21] Y. Ruan, H. Dong, A. Wang, *et al.*, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. ICLR*, 2024.

[22] L. Moreau, P. Groth, J. Cheney, T. Lebo, and S. Miles, "The rationale of PROV," *Journal of Web Semantics*, vol. 35, pp. 235-257, 2015.

[23] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159-176, 1999.

[24] B. Green, "The flaws of policies requiring human oversight of government algorithms," *Computer Law & Security Review*, vol. 45, art. 105681, 2022.

[25] J. Cobbe, M. S. A. Lee, and J. Singh, "Reviewable automated decision-making: A framework for accountable algorithmic systems," in *Proc. ACM FAccT*, 2021.